

Tea Shop

G13



Yasmin Itawi
1221426
Project manager



Suhaib Ewidat
1230290
Programmer



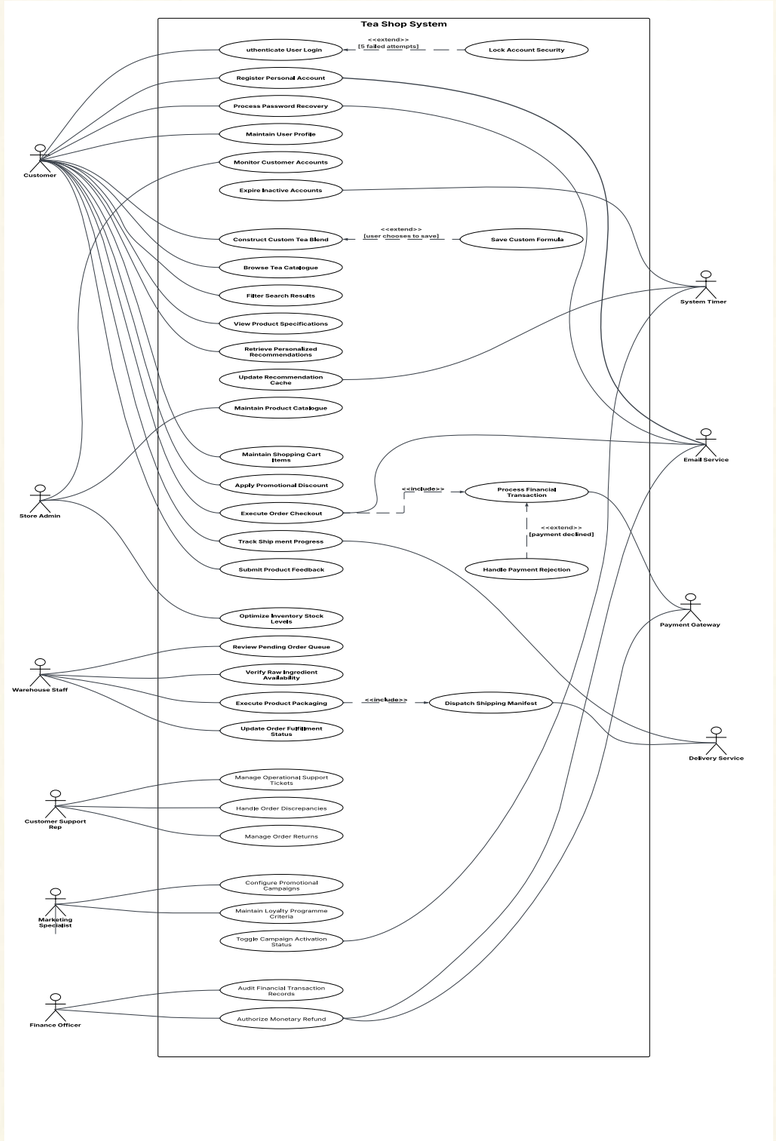
Reema Qashoo
1221009
Requirement Engineer

Business description

The Tea Shop System is an online platform for buying tea products. It allows customers to browse different types of tea, search for products, and order them easily from home. The system also gives customers the option to create their own tea blend by mixing different ingredients based on their taste.

In addition, the system provides personalized tea recommendations based on customer preferences and previous orders. Customers can also apply discount coupons and special offers during checkout. The platform supports secure payment methods, order tracking, and product reviews, making the shopping experience easier, safer, and more enjoyable.

Use case diagram



UC-19: Process Financial Transaction

Use Case Title	UC-19: Process Financial Transaction
Description	A Customer who has confirmed their order at the checkout screen initiates a payment request through the Tea Shop system. The system securely tokenises the payment details and forwards the transaction to an external PaymentGateway for authorisation. The gateway validates the card and returns an approval or rejection response. If approved, the system records the transaction, updates the order status to "Paid", generates a unique payment reference, and triggers a confirmation email via the EmailService. If declined, the system notifies the Customer and allows a retry with different payment details or order cancellation.
Actors	• Customer (Primary Actor): initiates the payment request. • PaymentGateway (Secondary Actor): external API that authorises or rejects the transaction. • EmailService (Secondary Actor): dispatches the payment confirmation email on success.
Data	• Payment method selected by Customer (credit card / debit card / digital wallet). • Tokenised card details (card number, expiry, CVV - never stored by the system). • Order total amount and currency. • PaymentGateway authorisation response code (approval / decline reason). • Generated unique payment reference number. • Order record identifier and current order status. • Customer email address used for confirmation dispatch.
Stimulus / Trigger	User command issued by the Customer upon clicking "Confirm & Pay" on the checkout screen, after UC-18 (Execute Order Checkout) has validated the cart contents and delivery address.
Pre-conditions	1. The Customer account record exists in the system with status Verified and the session is authenticated. 2. An order record exists in the system with status "Pending Payment", linked to the Customer's account, containing at least one item with confirmed stock availability. 3. The system has successfully completed UC-18 (Execute Order Checkout): cart contents are finalised and a valid delivery address is stored against the order record. 4. The Customer has selected a payment method on the checkout screen and the payment form is fully loaded.

UC-19: Process Financial Transaction

Workflow	(The use case starts when the Customer clicks "Confirm & Pay" on the checkout screen.) 1. The Customer reviews the final order total and selects a payment method (credit card, debit card, or digital wallet). 2. The Customer enters card details and clicks "Confirm Payment". 3. The system tokenises the card details and sends a secure authorisation request to the PaymentGateway API. 4. The PaymentGateway validates the card and checks available funds. 5. If authorised (Approved - Normal Path): 5.1 The system records the authorisation code and generates a unique payment reference number. 5.2 The system updates the order record status from "Pending Payment" to "Paid" in the database. 5.3 The system triggers the EmailService to dispatch an order confirmation receipt with the payment reference, order summary, and estimated delivery date. 5.4 The system redirects the Customer to the order confirmation page displaying the payment reference number. 5.5 The system adds the order to the Warehouse Staff pending fulfilment queue (UC-23). 6. Else, if declined (Alternative Path - UC-22: Handle Payment Rejection): 6.1 The PaymentGateway returns a rejection response with a decline reason code (e.g., insufficient funds, invalid card, timeout). 6.2 The system invokes UC-22 and displays a payment failure notification to the Customer with the decline reason. 6.3 The order record status remains "Pending Payment" in the database and no charge is made to the Customer's account. 6.4 If Customer selects "Retry": the sequence flow returns back to step 1 to allow entering different card details. 6.5 Else, if Customer selects "Cancel Order": the system updates the order status to "Cancelled" in the database, releases all reserved stock quantities back to active inventory, and returns the Customer to the shopping cart view. 7. Else, if PaymentGateway connection fails (Error Path 1): 7.1 The system fails to reach the PaymentGateway API due to a network or connectivity error. 7.2 The system logs the network failure with a timestamp and displays a technical connectivity error message to the Customer. 7.3 The order record status is strictly maintained as "Pending Payment" and no transaction log is created.

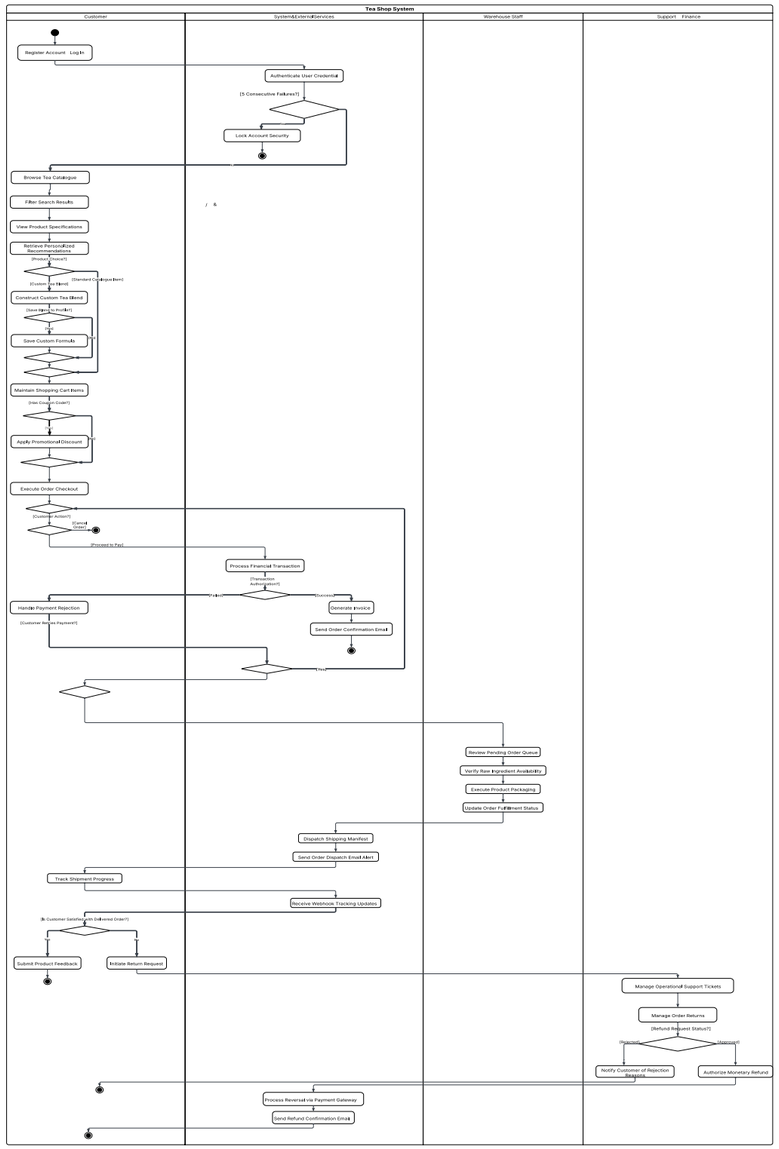
UC-19: Process Financial Transaction

	7.4 The system prompts the Customer to retry after a short interval or contact Customer Support. 8. Else, if database update fails after approval (Error Path 2): 8.1 The PaymentGateway has successfully approved the transaction, but the system encounters a critical database error while saving the record. 8.2 The system logs the database inconsistency and flags the order record status as "Pending Verification" to prevent duplicate charges. 8.3 The system records the payment reference number and displays an instruction notifying the Customer to contact Customer Support for manual confirmation. 8.4 The system automatically escalates the transaction metadata to the FinanceOfficer queue for manual resolution (UC-35).
Post-conditions / Response	Successful Payment Flow: 1. The order record status is updated to "Paid" in the database. 2. A unique payment reference number is securely stored and linked to the target order record. 3. The transaction is fully recorded in the payment audit log with a timestamp, amount, and gateway authorisation code. 4. The order entity is visible in the Warehouse Staff pending fulfilment queue. 5. A transaction receipt email has been successfully dispatched to the Customer via the EmailService. Unsuccessful Declined Flow: 1. The order record status remains strictly in the "Pending Payment" state within the database. 2. The specific PaymentGateway rejection code is appended to the order record history for audit purposes. 3. No monetary charge or transaction log entry is created against the Customer's financial account. Unsuccessful Cancelled by Customer: 1. The order record status is updated to "Cancelled" in the database. 2. Any previously reserved item quantities for this order are fully released back to the active inventory count. Unsuccessful Error Path 2 (DB Failure after Approval): 1. The order record status is set to "Pending Verification" in the database to halt automatic workflows. 2. A high-priority system discrepancy log entry is created and the task is pushed to the FinanceOfficer active dashboard for manual reconciliation.

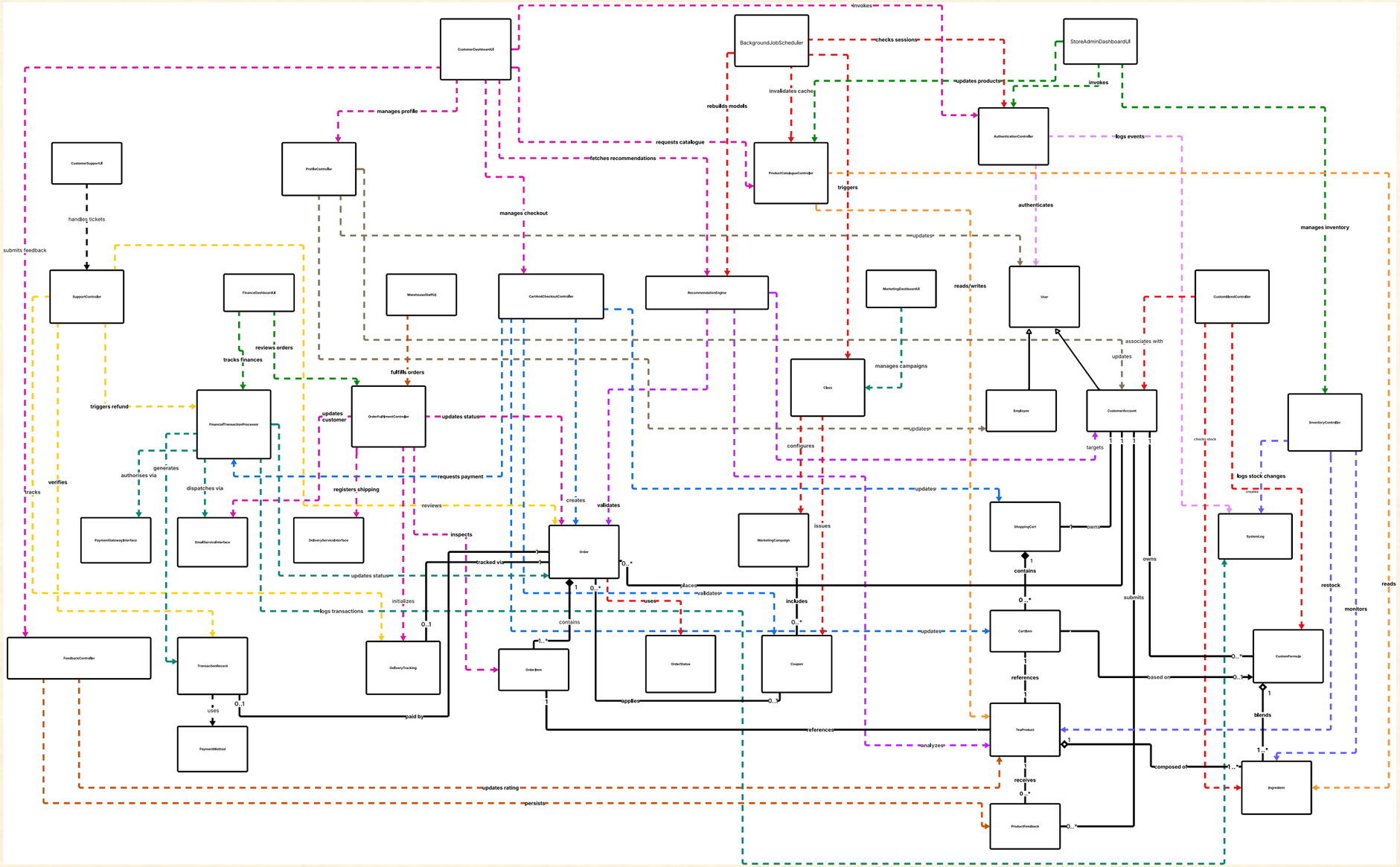
UC-19: Process Financial Transaction

Relationships	• <<include>> from UC-18 (Execute Order Checkout): UC-19 is always invoked as a mandatory sub-step of the checkout process. • <<extend>> UC-22 (Handle Payment Rejection): conditionally triggered when the PaymentGateway returns a declined response code.
Comments	The system must never store raw card data at any point; all card information is handled exclusively through tokenisation via the PaymentGateway. The system stores only the payment reference number and authorisation status. In Error Path 2, the FinanceOfficer must manually verify the transaction against gateway records to prevent duplicate charges. The CustomerSupportRep may initiate a refund via the PaymentGateway integration if a return is subsequently approved (UC-34, UC-36).

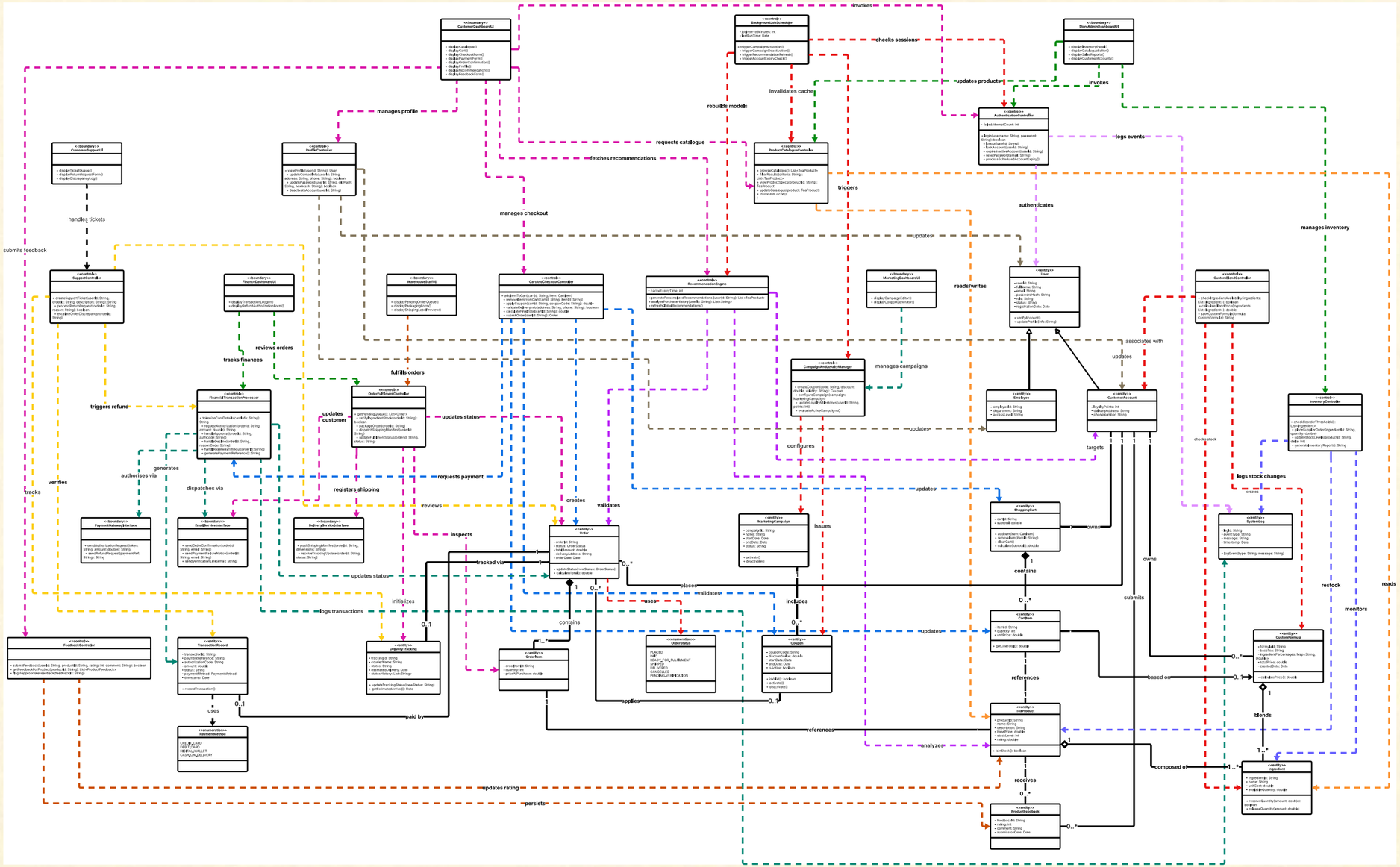
Activity diagram



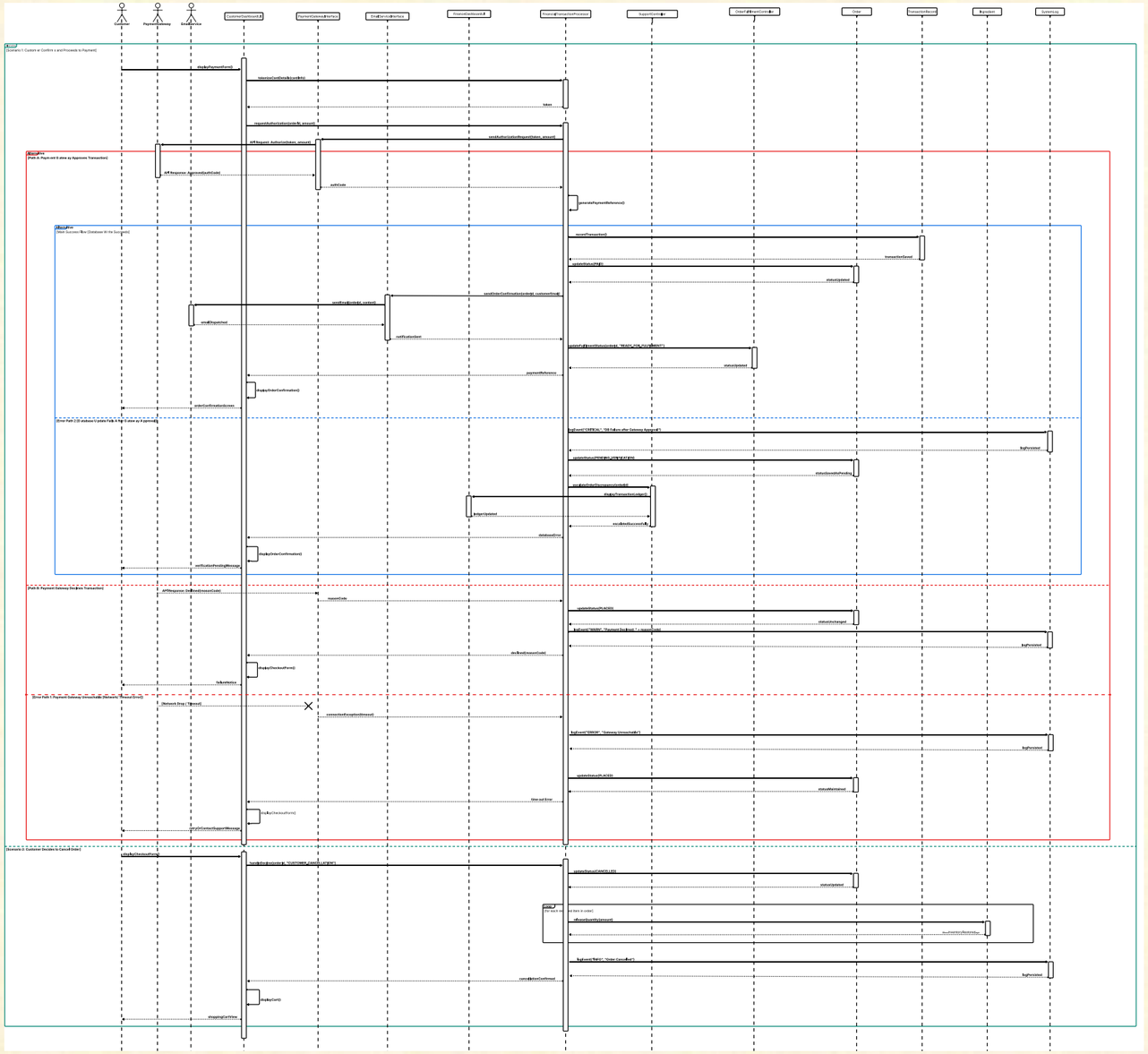
Analysis Class diagram



Detailed Class diagram



Sequence diagram



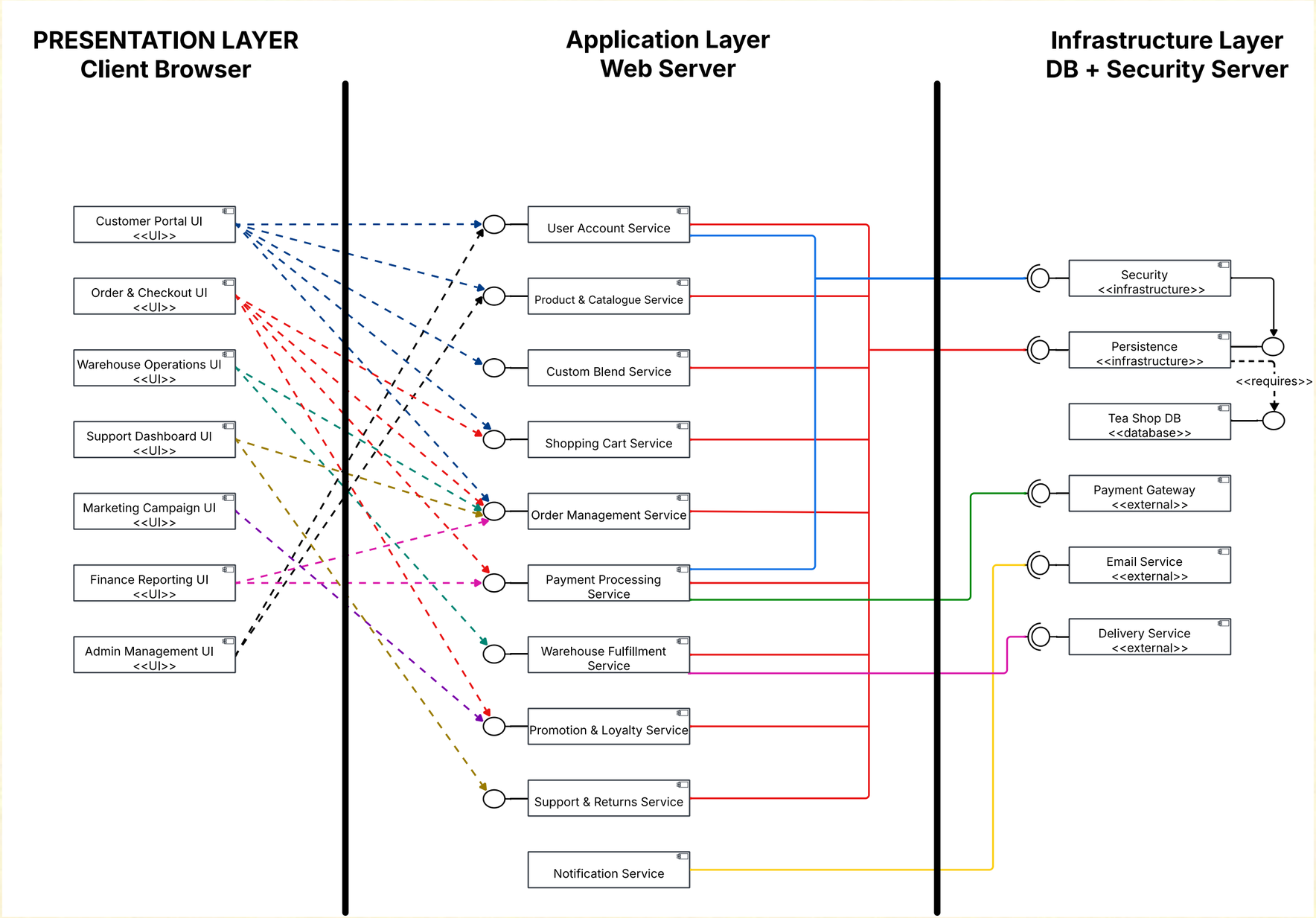
Design Goals

High Cohesion & Low Coupling
Each subsystem should have a clear and specific responsibility while minimizing dependencies between modules. This improves maintainability and allows different team members to work on separate parts without conflicts.

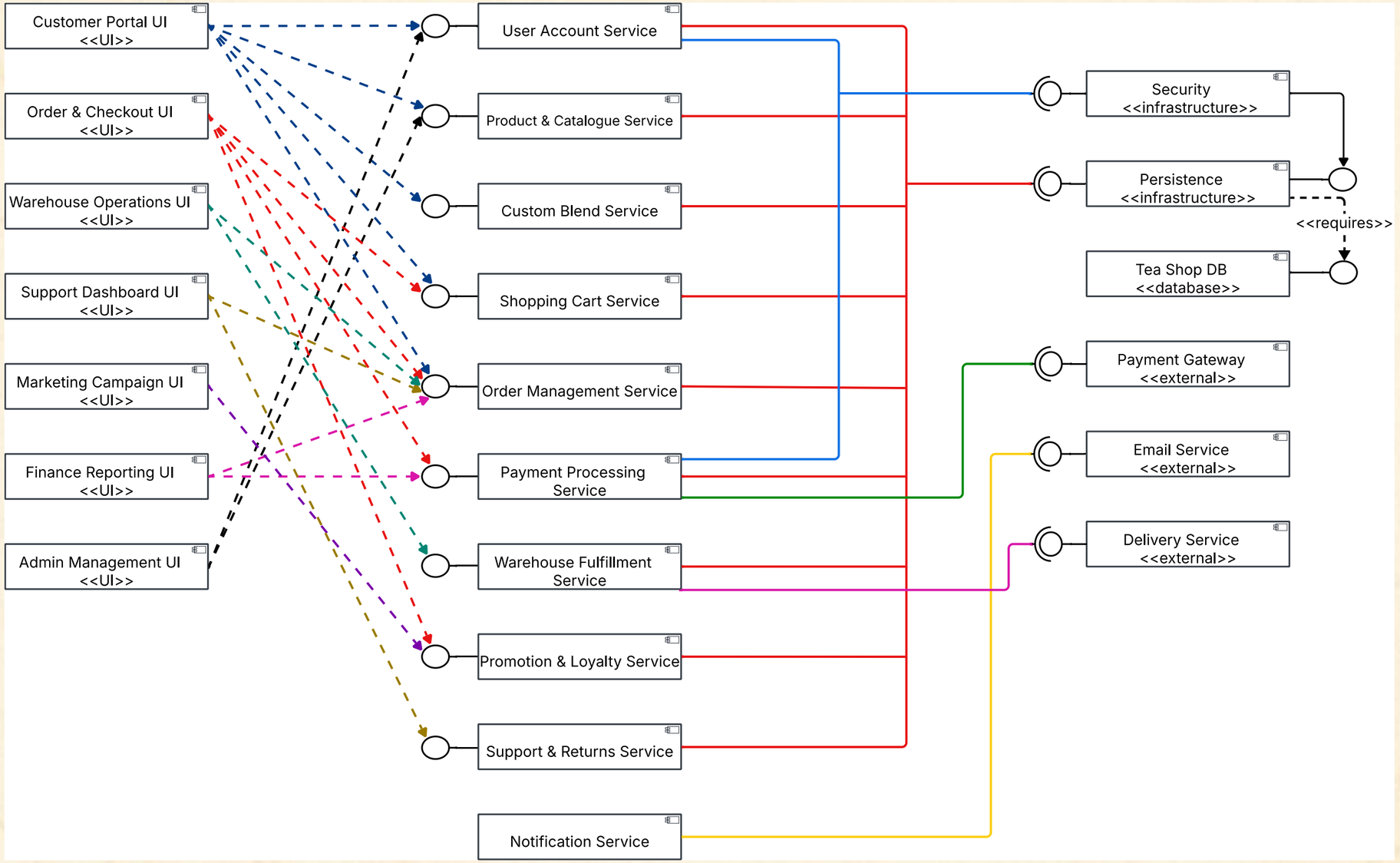
Extensibility & Modularity
The system should be easy to extend or update without changing the core business logic. External services such as payment gateways or notification systems can be replaced or upgraded easily through modular design.

High Reliability & Data Integrity
The system must ensure safe and consistent transactions, especially during payment processes. If a failure happens during payment, the system detects the issue, logs it, and prevents data loss or inconsistent order records.

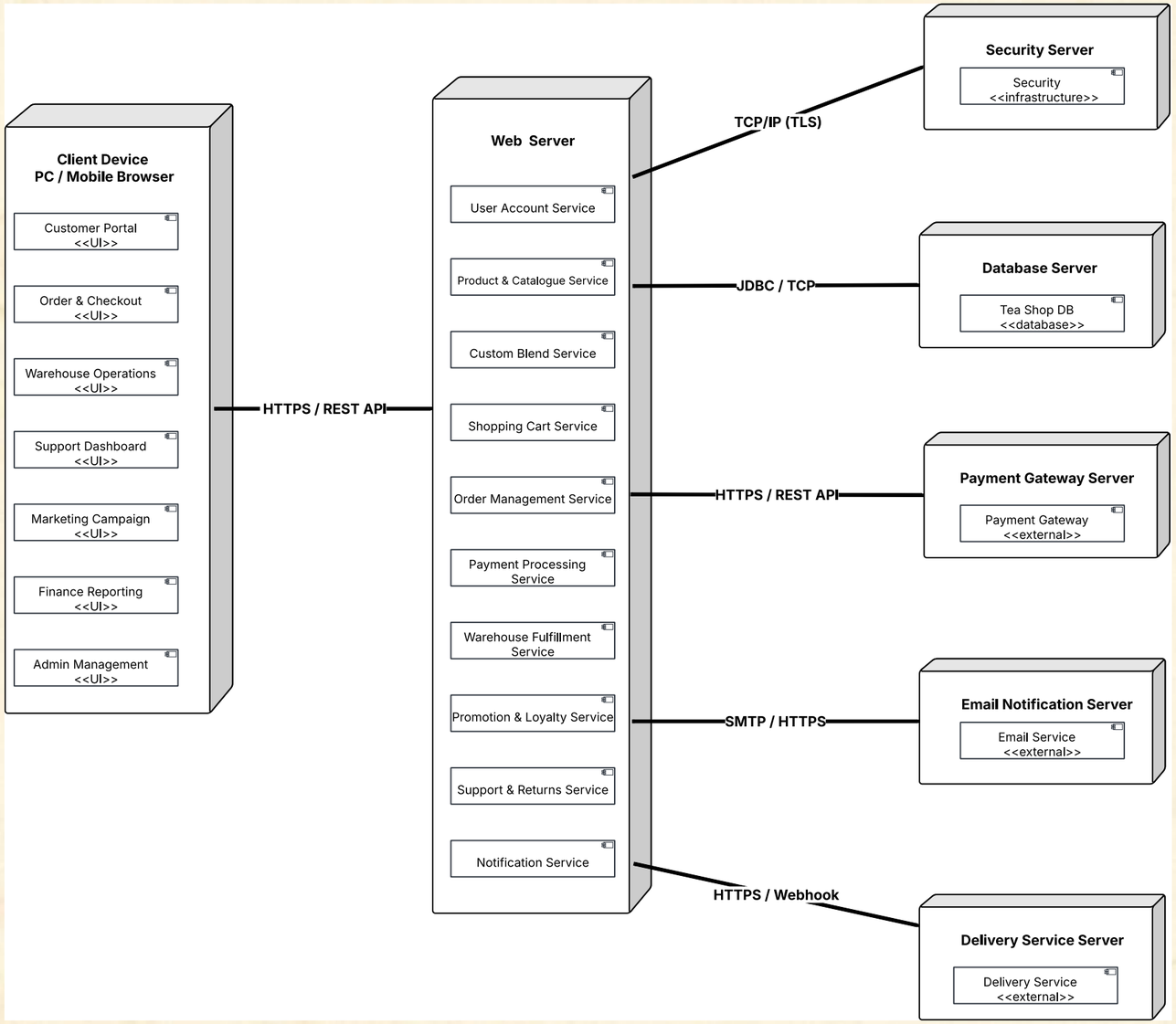
Architecture diagram



Component diagram



Deployment diagram



You can access all the project diagrams via this link:

https://lucid.app/lucidchart/1bcffec1-4332-4294-8f13-bf8aa22e7685/edit?viewport_loc=-655%2C-1514%2C3692%2C1696%2Cu_r4o7ypPy1b&invitationId=inv_1f5c88f9-fded-4d8e-b248-74de488179bf